

DESAFIO MÃO NA MASSA

Contexto: Vamos implementar o endpoint completo de gravação de um "Livro" em nosso sistema de biblioteca. Precisamos unir recebimento seguro de strings HTTP, tratamento lógico, acionamento do banco e o fluxo correto de redirecionamento.

Enunciado: Construa uma rota Flask chamada `/novo_livro` que receba dados de formulário via método POST, faça a conversão segura do número de páginas do livro e delegue a gravação a um DAO.

Resultado Esperado: Ao juntar todos os passos, o estudante deve submeter o HTML com um número de páginas inválido e ver a página recarregar mostrando o erro via Flash Message. Ao submeter corretamente, deve ser redirecionado para a rota de listagem (que pode exibir os dados). O servidor em nenhum momento deve parar de rodar no terminal (crash).

Passo a Passo: Para tornar este desafio claro e fácil de seguir, vamos desconstruir o processo em um roteiro passo a passo simples e com instruções detalhadas. Vamos construir cada peça como se fosse um quebra-cabeça.

Passo 1: Preparar a Base (O Arquivo `app.py`)

Vamos começar criando o "cérebro" da sua aplicação e a sua memória temporária.

1. **Crie um arquivo principal:** Chame-o de `app.py`.
2. **Traga as ferramentas:** No topo do arquivo, adicione as importações do Flask necessárias: `from flask import Flask, request, redirect, url_for, render_template, flash`.
3. **Ligue o motor:** Inicialize o Flask com `app = Flask(__name__)` e defina uma chave de segurança `app.secret_key = "chave_secreta"` (isso é obrigatório para podermos enviar mensagens de erro ao usuário mais tarde).
4. **Crie as estruturas de dados:**
 - Escreva uma classe `Livro` simples com o método `__init__` que recebe `titulo`, `autor` e `paginas`.
 - Escreva uma classe `LivroDAO` (o simulador de banco de dados). Dentro dela, crie uma lista vazia `banco_de_dados = []` para guardar os dados, e dois métodos básicos: `salvar(livro)` e `listar()`.

Passo 2: Criar a "Boca do Funil" (O Formulário)

Agora precisamos da tela onde o usuário vai digitar os dados.

1. **Crie a pasta de telas:** Na mesma pasta do seu `app.py`, crie uma pasta chamada `templates`.
2. **Crie o arquivo HTML:** Dentro de `templates`, crie o arquivo `formulario.html`.
3. **Configure o Formulário (`<form>`):**
 - A tag form deve ter `method="POST"` (para enviar os dados de forma oculta e segura).
 - Deve ter `action="/novo_livro"` (este é o endereço do servidor para onde o "pacote de dados" vai ser enviado).
4. **Os Atributos Críticos:** Para cada campo de texto (Título, Autor, Páginas), garanta que você colocou o atributo `name`. Por exemplo: `<input type="text" name="paginas">`. *Detalhe crucial: O Python vai procurar exatamente pela palavra que você escrever no atributo `name`.*

Passo 3: O Funil de Tratamento (A Rota no Servidor)

Esta é a parte principal do desafio. Volte ao arquivo `app.py` para construirmos a rota que recebe e limpa os dados.

1. **Abra a rota:** Escreva `@app.route('/novo_livro', methods=['POST'])`. Apenas métodos POST podem entrar aqui.

2. **Capture os dados brutos:** Extraia a informação do formulário usando o comando `request.form.get('nome_do_campo')`. Extraia o título, o autor e as páginas para variáveis separadas.
3. **Monte a Malha do Filtro (Try/Except):**
 - Abra um bloco `try`:
 - Tente converter a variável das páginas (que chegou como texto) em um número inteiro usando a função `int()`.
 - Adicione uma regra extra: se o número for menor ou igual a zero, levante um erro (`raise ValueError`).
4. **A Válvula de Escape (Except):**
 - Logo abaixo, crie o bloco `except ValueError`:
 - Se a conversão falhar (porque o usuário digitou letras, por exemplo), use o comando `flash("Mensagem de erro")`.
 - Use `return redirect(url_for('nome_da_funcao_do_formulario'))` para devolver o usuário ao formulário inicial para que ele tente novamente.

Passo 4: Salvar e Desviar o Fluxo (O Padrão PRG)

Ainda dentro da mesma rota do Passo 3, mas fora e abaixo do bloco Try/Except (ou seja, os dados sobreviveram ao filtro e estão puros).

1. **Empacotar:** Use os dados limpos para instanciar o seu objeto: `novo_livro = Livro(titulo, autor, paginas)`.
2. **Salvar no Banco:** Chame a sua classe DAO e use o método `salvar(novo_livro)`.
3. **O Redirecionamento (Passo fundamental):** Não devolva uma página HTML diretamente. Feche a operação com um redirecionamento limpo: `return redirect(url_for('nome_da_funcao_de_listagem'))`. Isso obriga o navegador do usuário a fazer uma nova requisição, "esquecendo" os dados do formulário anterior.

Passo 5: O Destino Final (A Lista)

Para provar que tudo funcionou, o usuário tem que ver o livro na biblioteca.

1. **A Rota da Lista:** No `app.py`, crie uma rota simples (por exemplo, `@app.route('/lista')`) que apenas acessa o DAO, busca todos os livros e os envia para um arquivo usando `render_template('lista.html', livros=meus_livros)`.
2. **O Arquivo Final:** Na pasta `templates`, crie o `lista.html`. Construa uma tabela em HTML simples e utilize um laço de repetição (`{% for livro in livros %}`) para desenhar uma nova linha na tabela para cada livro salvo.

Como fazer o "Teste de Fogo" final:

Com o código rodando, vá ao seu navegador e siga estes passos exatamente nesta ordem:

- **Teste o erro:** Tente adicionar um livro e escreva "duzentas" no campo de páginas. O servidor não pode quebrar (nada de erros 500 no navegador). O formulário deve recarregar e mostrar a sua mensagem de aviso de que apenas números são permitidos.
- **Teste o sucesso e a segurança:** Cadastre o livro corretamente, com o número 200. Você será redirecionado para a lista e verá o livro lá. Agora, **aperte a tecla F5 (Atualizar) do seu navegador**. Note que o navegador atualiza a página de forma tranquila, sem aquela janela chata perguntando se deseja "Reenviar os dados do formulário". Isso prova que o seu Post-Redirect-Get está perfeito!